

Name : yash Mahajan

Roll No : 26331012

Division : A

Assignment - I

classmate

Date _____

Page _____

Operating System (OS) & Types of operating System

• Definition of operating System :

An operating System (OS) is System Software that acts as an interface between the user and the computer hardware. It manages hardware resources, executes programs, controls memory, files, input/output devices, and provides a platform for application software to run efficiently.

(1) Batch Operating System

Definition :

A Batch operating System is an operating System that collects similar jobs or tasks into a group called batches and processes them one after another automatically. The user does not interact with the computer while the jobs are being executed. Once a batch is submitted, the operating System completes all the jobs in sequence without any manual intervention. It is mainly used for repetitive and large scale processing tasks.

— Advantages :

- Efficient for processing a large number of similar jobs
- Reduces CPU idle time
- Requires very little user intervention
- Suitable for repetitive tasks
- Improves overall system utilization.

- Disadvantages

- Not direct interaction between the user and the system
- Long waiting time for job completion
- Difficult to identify and correct errors
- Urgent jobs cannot be executed immediately
- Not suitable for interactive applications

- Usage

- Payroll processing
- Bank statement generation
- Electricity bill generation
- Examination result processing
- Large data processing tasks

(2) Multiprogramming Operating System

Definition :

A Multiprogramming Operating System is an operating system that keeps multiple programs in the computer's main memory at the same time. When one program is waiting for an input or output operation, the CPU immediately switches to another program instead of remaining idle. This improves CPU utilization, increases system efficiency, and allows several programs to be executed simultaneously.

(4) Information Maintenance System calls:

These System calls are used to provide information about processes, file & the system.

— Example: `getpid()`, `getppid()`, `stat()`, `lstat()`, `getpriority()`

— Functions:

- (1) Get process ID
- (2) Get parent process ID
- (3) Retrieve file information
- (4) Check process priority
- (5) Obtain system-related information

(5) Communication System calls:

These System calls enable Communication between processes and monitor input/output operations.

— Examples: `Select()`

— Functions:

- (1) Exchange information between processes
- (2) Monitor multiple file descriptors
- (3) Handle I/O events efficiently.

(6) Memory management System calls:

These System calls manage the memory used by processes.

— Examples: `brk()`, `sbrk()`, `mmap()`, `munmap()`, `mprotect()`

— Functions:

- (1) Allocate memory
- (2) Release memory
- (3) Map files into memory
- (4) change memory protection
- (5) Manage virtual memory.

Page _____

and allocates them to different tasks. This creates the impression that all programs are running simultaneously while ensuring quick response & efficient utilization of system resources.

— Advantages

- Fast response time
- Allows multiple applications to run simultaneously
- Improves user productivity.
- Efficient utilization of CPU resources
- Supports multiple users.

— Disadvantages

- Requires more memory
- Performance decreases if many programs run simultaneously.
- Security issues many arise
- more complex single-task operating system
- Requires efficient CPU scheduling.

— Usage

- windows operating system
- Linux operating system
- Mac OS
- office computers.
- Educational institutions.

(4) Multiprocessing operating system :

- Definition: A multiprocessing operating system is an operating system that uses two or more processors to execute multiple tasks simultaneously, resulting in faster processing and improved performance.

- Advantages :

- (1) Multiple processors execute tasks at the same time.
- (2) Reduces the time needed to complete programs.
- (3) If one processor fails, others may continue work.

- Disadvantages :

- (1) Requires multiple processors and specialized hardware.
- (2) More difficult to develop and manage.
- (3) Uses more electricity than a single-processor system.

- Uses :

- (1) Scientific research and simulations.
- (2) Large database servers.
- (3) Cloud computing and data centers.

(5) Real-time operating system

- Definition: A real-time operating system processes data and responds within a fixed time limit.

- Advantages :

- (1) Very fast response.
- (2) High reliability.
- (3) Accurate timing.

— Disadvantages :

- (1) Expensive to develop
- (2) Limited multitasking
- (3) Complex programming.

— Uses :

- (1) Medical equipment
- (2) Air traffic Control
- (3) Robotics.
- (6) Distributed operating System

• Definition: A Distributed operating System manages a group of connected computers and makes them appear as a single system.

— Advantages :

- (1) Resource sharing
- (2) High reliability
- (3) Faster processing through workload distribution

— Disadvantages :

- (1) Complex to design and maintain
- (2) Network failure can affect performance
- (3) Higher setup cost.

— Uses :

- (1) Cloud computing
- (2) Scientific research
- (3) Large business networks.

(7) Network operating System:

• **Definition:** A Network operating System manages network resources and enables communication between computers connected through a network.

— Advantages:

- (1) Easy File and printer sharing
- (2) Centralized management.
- (3) Better Security.

— Disadvantages:

- (1) Expensive Server Setup
- (2) Requires skilled administrators
- (3) Server Failure can affect all users.

— Uses:

- (1) offices
- (2) Schools
- (3) Corporate networks

Q.2) Define the types of System Calls

— **Types of System Calls:** A System Call is a request made by a user program to the operating system kernel to perform a service. System calls are mainly classified into six types.

(1) Process Control System Calls:

These System Calls are used to create, execute, terminate and manage processes.

— Examples: `fork()`, `execve()`, `exit()`, `wait()`, `waitpid()`, `kill()`

— Functions:

- (1) Create a new process
- (2) Execute a program
- (3) Terminate a process
- (4) wait for child processes
- (5) Send signals to processes.

(2) File management System calls:

These System Calls are used to create, open, read, write, close, rename and delete files.

— Examples: `open()`, `create()`, `read()`, `write()`, `close()`, `rename()`, `unlink()`.

— Functions:

- (1) Create files:
- (2) open and close files.
- (3) Read and write data
- (4) Delete or rename files
- (5) Move the file pointer

(3) Device Management System Calls:

These System Calls are used to control and communicate with hardware devices.

— Example: `ioctl()`, `read()`, `write()`

— Functions:

- (1) Access hardware devices
- (2) Configure device settings
- (3) Read from and write to device.

Q3) write the Definitions of following list

- (1) `fork()`: Create a new child process by duplicating the calling process.
- (2) `vfork()`: Create a child process that shares the parent's address space until it calls `execve()` or `exit()`.
- (3) `clone()`: Create a new process or a thread with shared resources based on specified flags.
- (4) `execve()`: Replace the current process image with a new program.
- (5) `exit()`: Terminates a process after performing cleanup operations.
- (6) `_exit()`: Immediately terminates a process without performing standard cleanup operations.
- (7) `wait()`: waits until one child process terminates.
- (8) `waitpid()`: waits for a specific child process to finish.
- (9) `waitid()`: waits for child process state changes & returns detailed information.
- (10) `kill()`: Send a signal to a process or process group.

- (1) `getpid()` : Returns the process ID (PID) of the calling process.
- (2) `getppid()` : Returns the parent process ID (PPID).
- (3) `nice()` : Changes the scheduling priority of the current process.
- (4) `setpriority()` : Set the priority of a process, process group or user.
- (5) `getpriority()` : Returns the scheduling priority of process.
- (6) `open()` : opens an existing file or creates one if requested & returns a file descriptor.
- (7) `creat()` : create a new file or truncates an existing file.
- (8) `close()` : close an opened file descriptor.
- (9) `read()` : Reads data from a file descriptor into memory.
- (10) `write()` : writes data from memory to a file descriptor.
- (11) `pread()` : Reads data from a specified file offset without changing the current file position.

- (22) `pwrite`: Write data or a specified file offset without changing the current file position.
- (23) `lseek()`: Moves the file pointer to a specified position.
- (24) `stat()`: Returns information about a file.
- (25) `fstat()`: Returns information about a file using its file descriptor.
- (26) `lstat()`: Returns information about a symbolic link instead of the target file.
- (27) `access`: Checks whether the process has permission to access a file.
- (28) `unlink()`: Delete a file.
- (29) `rename()`: Changes the name or location of a file or directory.
- (30) `mkdir()`: Create a new directory.
- (31) `rmdir()`: Removes an empty directory.
- (32) `dup()`: Duplicates an existing file descriptor.
- (33) `dup2()`: Duplicates a file descriptor to a specified descriptor number.

- (34) `fcntl()`: Performs various operations on file descriptors such as looking & changing flags.
- (35) `fsync()`: Write all buffered file data & metadata to disk.
- (36) `fdatasync()`: Writes buffered file data to disk but minimizes metadata updates.
- (37) `ftruncate()`: changes the size of a file by its name.
- (38) `ftruncate()`: changes the size of a file using its file descriptor.
- (39) `select()`: Monitors multiple file descriptors & waits until one becomes ready for I/O.
- (40) `ioctl()`: performs device-specific input/output control operations.
- (41) `poll()`: Monitors multiple file descriptors for input output or error events.
- (42) `brk()`: changes the end of the process's data segment.
- (43) `sbrk()`: Increases or decreases the process heap.
- (44) `mmap()`: Maps a file or anonymous memory into the process address space.

- (45) `munmap()`: Remove a memory mapping created by `mmap()`.
- (46) `mprotect()`: changes the access permissions of a memory region.
- (47) `mlock()`: locks memory pages into RAM to prevent swapping.
- (48) `munlock()`: unlocks previously locked memory pages.
- (49) `msync()`: synchronizes a mapped memory region with the underlying storage.
- (50) `madvise()`: provide advice to the kernel about the expected use of a memory region.